

The Zenlang Language Specification

Table of Contents

1. Introduction & Language Philosophy

1.1 Core Guiding Principles

2. Lexical Grammar & Source Representation

2.1 Comments & Documentation

2.2 Identifiers & Lexical Scopes

3. The Zen Trinity: Code, Data, and Mark

4. Types, Literals & Sentinels

4.1 First-Class Capital Sentinels

4.2 Primitive & Collection Literals

5. Variables, Assignment & Flow Operators

5.1 Binding (let) and Mutation (set)

6. Operators & Expressions

7. Control Flow & Directional Guards

7.1 Branching with when / or

7.2 Single-Line Guard Returns

7.3 Iteration Loops

8. Functions & Snapshot Closures

8.1 Concise Expression Functions

9. Data Structures & Pattern Matching

10. Object-Oriented Programming & Classes

11. Error Handling & Source-Mapped Diagnostics

12. Module System & Standard Library

13. Unified Task Concurrency & Memory Arenas

14. Three-Layer Architecture & Native Selfhost

1. Introduction & Language Philosophy

Zenlang is a modern, optionally typed, Unix-native programming language designed for clarity, visual simplicity, and high-performance execution across scripts, terminal user interfaces, and networked backend services.

The Dual Execution Guarantee

Every valid Zenlang program executes with byte-exact behavioral equivalence in interpreted VM mode and Ahead-Of-Time (-g) native C compilation.

1.1 Core Guiding Principles

1. Visual Data Flow: Data movement is explicit in the code syntax. Variable binding and assignment use `->` (moving left-to-right into the target), while function returns, yields, and emissions use `<-` (returning outward to the caller).
2. Explicitness Over Magic: No hidden type coercions, no implicit variable captures, and no silent conversions. Absence is represented by `Nothing`, and type zero-states are represented by `Default`.
3. Dual Execution Parity: Every valid Zenlang program (.zl) executes identically in two complementary modes without code changes:

- Interpreted Script & Bytecode Mode: Direct AST/bytecode execution for instantaneous feedback and rapid iteration.

- Native Compiled Mode (-g): Ahead-Of-Time (AOT) transpilation to optimized C and native machine code, linking with the shared runtime in runtime/.
4. Terminal as a First-Class Citizen: Sub-millisecond ANSI canvas rendering, 60 FPS redraw capabilities, interactive keyboard polling, and declarative table formatting are standard library primitives.
5. Snapshot-by-Value Concurrency: Captured outer variables are snapshotted by value at instantiation time, eliminating data races without heavy runtime locks.
6. Pure Native Self-Hosting Architecture: The compiler is written in Zenlang and compiles itself into a standalone native C binary with zero Python dependencies.

Production Deployment

For performance-critical networked services or CLI utilities, compile with `zen compile main.zl -o app` to produce a zero-dependency standalone ELF machine binary.

Concept	Syntax Example	Semantic Guarantee
Assignment	<code>let x -> 42</code>	Visual flow from expression into binding
Return	<code><- result</code>	Visual flow from function back to caller
Guard Exit	<code>when error <- 1</code>	Concise single-line early return
Sentinels	<code>Nothing, Default</code>	Explicit absent and zero states
Range	<code>0 .. 10, 0 ..= 10</code>	Exclusive and inclusive numeric intervals
Pattern Matching	<code>when val is [a, b]</code>	Zero-overhead structural destructuring

2. Lexical Grammar & Source Representation

Zenlang source files (.zl) are UTF-8 encoded text streams. Whitespace separates tokens and has no semantic significance except within string literals and comments. Indentation is stylistic and not syntactically enforced.

2.1 Comments & Documentation

Zenlang supports three distinct comment forms: single-line comments (`//`), multi-line block comments (`/* ... */`), and formal doc-comments (`/*! ... !/`). Doc-comments are introspected by compiler tooling (`zendoc`) to produce structured API reference manuals.

```
// Single-line comment for brief annotations [zen]
let x -> 42

/*
 * Multi-line comment for detailed
 * algorithmic documentation.
 */

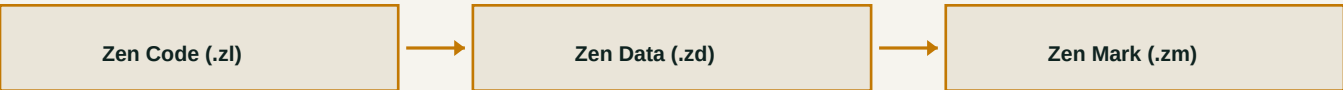
/*!
 Calculates the factorial of an integer.
 @param n Integer The input value
 @return Integer The calculated factorial
 !/
function factorial(n: Integer) : Integer {
  when n <= 1 <- 1
  <- n * factorial(n - 1)
}
```

2.2 Identifiers & Lexical Scopes

Identifiers begin with an ASCII letter (a-z, A-Z) or underscore (`_`), followed by alphanumeric characters. Identifiers are strictly case-sensitive. Code blocks delimited by curly braces (`{ ... }`) create nested lexical scopes where inner scopes inherit outer bindings.

3. The Zen Trinity: Code, Data, and Mark

Zenlang unifies computation, data serialization, and technical documentation into three deeply cohesive, mutually reinforcing formats known as The Zen Trinity:



Pillar	Extension	Canonical Module	Core Purpose
Zen Code	.zl	zen.tooling.zencode	Universal executable programming language (V
Zen Data	.zd	zen.data.zendata	Declarative data serialization superseding JSON
Zen Mark	.zm	zen.text.zenmark	Document Object Model and publishing engine

Shared Trinity Semantics

All three Trinity formats share identical token semantics, flow arrows (->), sentinels (Nothing, Default), and raw backtick string escaping.

4. Types, Literals & Sentinels

Zenlang is optionally typed. You may declare types explicitly or allow the bidirectional type inference engine to resolve types at compile time.

4.1 First-Class Capital Sentinels

Zenlang defines foundational sentinels with capitalized canonical spellings:

- Nothing: Represents the complete absence of a value (assignable and comparable: `val == Nothing`).
- Default: Represents the zero-state of a type (0 for integers, 0.0 for decimals, false for booleans, empty string for strings, and empty list/map for collections).
- True / False: Standard boolean truth values.

```
let optional_user -> Nothing    // Explicit absent value [zen]
let empty_buffer  -> Default    // Type zero-state (empty string, 0, [], {})
let is_ready      -> True       // Boolean True
let is_disabled   -> False      // Boolean False

// Sentinel comparisons
when optional_user == Nothing {
    // Handle absent case
}
```

4.2 Primitive & Collection Literals

Zenlang provides expressive literal syntax for numbers, strings, lists, and flow maps:

```
// Numeric Literals
let decimal_int -> 42
let hex_val     -> 0x2A
let binary_val  -> 0b101010
let float_val   -> 3.14159

// String Literals
let simple_str  -> "Standard double-quoted string with \n escapes"
let raw_str     -> 'Raw backtick string with no escaping needed'

// Collections
let numbers_list -> [1, 2, 3, 4, 5]
let config_map   -> {
  'host' -> '127.0.0.1',
  'port' -> 8080,
  'active' -> True
}
```

5. Variables, Assignment & Flow Operators

Zenlang uses directional flow arrows to make data movement visually unambiguous across all statements.

5.1 Binding (let) and Mutation (set)

Variables are introduced into scope using the `let` keyword followed by an identifier, the right arrow (`->`), and an initializing expression. Subsequent reassignments use the `set` keyword to make mutability intentional and explicit.

```
let count -> 0           // Variable binding with initial value
set count -> count + 1    // Reassignment using set keyword

// Destructuring assignment
let [first, second] -> [10, 20]
let { 'name' -> user_name, 'role' -> user_role } -> user_record
```

6. Operators & Expressions

Zenlang features a comprehensive suite of operators with well-defined precedence levels:

Category	Operators	Description
Flow	-> , <-	Assignment flow into target, return flow out of scope
Arithmetic	+, -, *, / , %	Standard numeric arithmetic and string concatenation
Comparison	== , != , < , <= , > , >=	Value equality and relational ordering
Logical	and , or , not	Short-circuiting boolean operations
Range	.. , ..=	Half-open [a, b) and closed [a, b] ranges
Extended	++ , --	In-place increment and decrement

7. Control Flow & Directional Guards

Control flow in Zenlang is unified around structured branching and iteration constructs designed for readability.

7.1 Branching with when / or

Conditional branching replaces traditional if/else chains with the cleaner when / or construct:

```
when status_code == 200 {
    io.writeln("Request succeeded")
} or when status_code == 404 {
    io.writeln("Resource not found")
} or {
    io.writeln("Unexpected status code")
}
```

[zen]

7.2 Single-Line Guard Returns

To eliminate deep nesting and simplify error handling, Zenlang allows single-line guard exits with when condition <- return_value:

```
function validate_age(age) {
    when age < 0 <- { 'ok' -> False, 'error' -> 'Age cannot be negative' }
    when age > 150 <- { 'ok' -> False, 'error' -> 'Age exceeds biological limits' }

    // Main logic executes only when all guards pass
    <- { 'ok' -> True, 'value' -> age }
}
```

[zen]

7.3 Iteration Loops

Zenlang provides three distinct loop structures: do while, do until, and collection iteration with do for item in collection:

```
// 1. While Loop [zen]
let i -> 0
do while i < 5 {
  io.writeln("Index: " + Str.to_string(i))
  set i -> i + 1
}

// 2. Collection Iteration
let fruits -> ["Apple", "Banana", "Cherry"]
do for fruit in fruits {
  io.writeln("Fruit: " + fruit)
}
```

8. Functions & Snapshot Closures

Functions are first-class values in Zenlang. Functions accept parameters, declare optional return types, and emit return values using the left arrow (<-).

8.1 Concise Expression Functions

Short single-expression functions can be written on a single line using function name(params) <- expr:

```
// Concise single-expression functions [zen]
function square(x) <- x * x
function add(a, b) <- a + b
function is_even(n) <- n % 2 == 0

// Higher-order function with capture-by-value closure
function make_multiplier(factor) {
  // factor is snapshotted by value
  <- function(x) <- x * factor
}
```

9. Data Structures & Pattern Matching

Zenlang provides lightweight tagged data structures with the structure keyword and exhaustive pattern matching on variants:

```
structure Point {
  x: Integer
  y: Integer
}

structure Rect {
  origin: Point
  width: Integer
  height: Integer
}

// Instantiation
let p -> Point { x -> 10, y -> 20 }
let r -> Rect { origin -> p, width -> 100, height -> 50 }

// Pattern matching and destructuring
when r is Rect { origin -> Point { x, y }, width, height } {
  io.writeln("Origin: " + Str.to_string(x) + ", " + Str.to_string(y))
  io.writeln("Area: " + Str.to_string(width * height))
}
```

10. Object-Oriented Programming & Classes

Zenlang supports object-oriented modeling with classes, constructors, methods, and single inheritance:

```
class Animal {
  let name

  function init(name) {
    self.name -> name
  }

  function speak() {
    <- "Some generic animal sound"
  }
}

class Dog extends Animal {
  let breed

  function init(name, breed) {
    parent.init(name)
    self.breed -> breed
  }

  function speak() {
    <- "Woof! My name is " + self.name
  }
}
```

[zen]

11. Error Handling & Source-Mapped Diagnostics

Zenlang eliminates unhandled runtime crashes with explicit check / or error boundaries and Rust/Clang-style source-mapped caret diagnostics:

```
import zen.io.file as file
import zen.data.zendata as zd

function parse_config(filepath) {
  check {
    let data -> file.read(filepath)
    let cfg -> zd.parse(data)
    <- { 'ok' -> True, 'config' -> cfg }
  } or {
    <- { 'ok' -> False, 'error' -> 'Failed to load config from ' + filepath }
  }
}
```

[zen]

12. Module System & Standard Library

Zenlang provides clean hierarchical imports with the `import` keyword, supporting nested domain namespaces, module aliasing, and selective symbol import:

Import Pattern	Example	Semantic Effect
Domain Import	<code>import zen.io.io as io</code>	Binds the <code>io</code> module under an explicit alias
Aliased Import	<code>import zen.sys.term as term</code>	Binds the <code>term</code> module under an explicit alias
Selective Import	<code>from zen.math.math import sqrt, pow</code>	Imports individual functions into local scope
Trinity Aliases	<code>import zen.zenmark as zm</code>	Binds the standard Zen Mark engine

13. Unified Task Concurrency & Memory Arenas

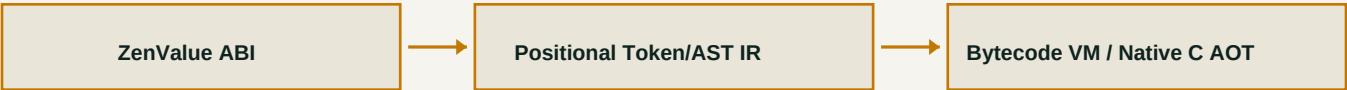
Zenlang integrates asynchronous tasks with chunk-chained bump memory arenas for deterministic, zero-GC execution:

```
import zen.memory as memory [zen]

function process_batch_in_arena(items) {
  // 32MB scoped arena allocated on entry, freed on block exit in 0(1)
  with memory.create_arena(32 * 1024 * 1024) as arena {
    do for item in items {
      // Memory allocated within this block lives inside the arena
      let processed -> item + "_processed"
      io.writeln(processed)
    }
  }
}
```

14. Three-Layer Architecture & Native Selfhost

Zenlang achieves maximum performance and cross-platform portability through its Three-Layer execution model:



Unified ZenValue ABI

Interpreted scripts can import and call pre-compiled C modules without wrapper glue code or foreign-function overhead because both layers utilize identical ZenValue struct layouts.

Execution Mode	Compiler Command	Target Environment	Primary Advantage
Interpreted / VM	zen script.zl	Development & Scripting	Instant sub-millisecond startup & dynamic reload
Native C AOT	zen compile main.zl -o app	Production Deployments	Bare-metal execution speed, minimal memory &
Native Selfhost	zen-selfhost compile main.zl out.c	Toolchain & Host	Self-contained compiler with zero external dependencies